Please enter the following command in your terminal:

```
npm install -g coffee-script
```

After that, issue the following command using the terminal:

```
coffee -v
```

It will show you the version of CoffeeScript installed.

Why CoffeeScript is good for you

There are plenty of other languages out there, so what makes CoffeeScript the 11th most popular language in Github? Why should you consider learning CoffeeScript? Those were exactly the questions I asked when I first heard about CoffeeScript.

Keep in mind that the sole purpose of CoffeeScript is to produce efficient JavaScript, without writing much code. CoffeeScript also focuses on highlighting all the good aspects of JavaScript with simple syntax.

Since CoffeeScript has been inspired by Python, Ruby and Haskell, it adopted the syntax and coding style from them, which makes it very unique and powerful. Also, CoffeeScript produces one-third the amount of script that the original JavaScript does. This means you can write a typical 'Hello world' program in a single line, whereas in JavaScript you have to write three lines. So now you can enjoy the simplicity of Ruby with the power of JavaScript.

CoffeeScript might come in handy when you're familiar with the basics of JavaScript, because only the syntax is different. But JavaScript is at the core of CoffeeScript, so it is advisable to learn the basics of JavaScript first.

Your first sip of CoffeeScript

Let's start with something small —a 'Hello world' programme will suffice for now. Open your favourite text editor and type the following line:

```
console.log 'Hello, World'
```

Save the document with the extension *hello.coffee*, then go to terminal, and change to the directory where the above script is stored. Run the above script in the terminal by issuing the following command:

```
coffee hello.coffee
```

You will see the greeting in your terminal. Now let us examine how to convert this CoffeeScript into its equivalent in JavaScript.

Type the following command in your terminal:

```
coffee -c hello.coffee
```

The flag *-c* stands for compile, which means it is compiling your CoffeeScript to JavaScript. You will find a *hello.js* file in the same directory you're in. When you open that JavaScript file, it will show you the compiled JavaScript:

```
(function() {

  console.log('Hello, world');

}).call(this);
```

There are several other useful options available for your CoffeeScript, but I'd like to focus on two options that could be a great help to you when you're working with CoffeeScript.

The option *-p* shows the compiled JavaScript on your terminal once you're done with writing your CoffeeScript. This could be useful when you want to peek into the desired JavaScript on the terminal but not on the separate JavaScript file, so that you don't populate your directory with a lot of files, unless you're satisfied with the desired output.

The option -w stands for 'watch', which allows you to keep an eye on things when you're making changes to your script. When you combine the *-c* (compile) option with *w* as *-cw*, CoffeeScript runs in the background and recompiles the source script as soon as you make changes. You don't have to manually recompile every time you make changes in your script. CoffeeScript will take care of it for you.

The REPL

Another interesting feature of CoffeeScript is REPL (Read-Evaluate-Print-Loop). Similar to Ruby's *irb* (interactive ruby), when you run CoffeeScript without any arguments in your terminal, the prompt changes to something like this:

```
coffee>
```

You can invoke the same by using the following options:

```
coffee -i
```

Or if you want something else, you can use:

```
coffee –interactive
```

This feature will be extremely useful when you want to evaluate something on the fly. If you want to deal with some expression whose output you are not so sure of, you could use it on the REPL and see what the actual output is.

Running CoffeeScript in your browser

CoffeeScript is capable of running anywhere JavaScript runs, which includes your browser!

Yes, you can run your CoffeeScript directly on the browser without compiling it to JavaScript, but first